

COMPSCI 701:
Creating Maintainable Software
Assessing Alterability

Yu-Cheng Tu, Ewan Tempero
School of Computer Science

Agenda

- Agenda

- Assignment 4
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- Coupling and Cohesion
- Code Smells
- Large Class
- Key Points

- Admin
 - ??
- Assignment 4
- Measurement
- Coupling and Cohesion
- Some Code Smells

Assignment 4

- Agenda
- **Assignment 4**
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- Coupling and Cohesion
- Code Smells
- Large Class
- Key Points

- The first rule of Assignment 4 is: **Do not merge to main branch**
- Review two Kalah implementations (“blue” — design1072 and “green” — design1078) with respect to aspects of their maintainability
- The reviews should be informed by the questions asked in the “report” (*quiz*) on Canvas.
- “Report” (Canvas quiz)
 1. Answer each question in the report.
 2. For many of the questions, write a **pull request comment** for the relevant part of the implementation that provides your answer.
 3. Paste the link to the pull request comment in the associated question in the survey
- Practically speaking you may prefer to review the implementations first (being aware of the questions) before completing the reports.

Assignment 4

- Agenda
- **Assignment 4**
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- Coupling and Cohesion
- Code Smells
- Large Class
- Key Points

- **Do not merge to main branch**
- You should be able to work on the report incrementally until the due date (no need to finish it all at once). **please confirm**
- Due 23:59 Friday 24 October (end of week 12).
- You must submit your quiz before the **CANVAS due date**. You will not be able to change them after this point.
- **Do not merge to main branch**

- Agenda
- **Assignment 4**
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- Coupling and Cohesion
- Code Smells
- Large Class
- Key Points

Potential Assessment Question

- I should not merge to the main branch for assignment 4.
 - (a) Agree
 - (b) Strongly agree
 - (c) Very Strongly agree
 - (d) Absolutely positively very strongly agree
 - (e) All of the above

- Agenda
- Assignment 4
- **Assessing Alterability**
- Previously
- Measurement
- Measuring Quality
- Coupling and Cohesion
- Code Smells
- Large Class
- Key Points

Assessing Alterability (or any Quality Attribute)

- Any form of assessment is about **comparison**
- Comparison can be difficult or expensive to do
- **Measurement** provides a simpler and cheaper way to do comparison
- ... **provided the measurement is valid**

- Agenda
- Assignment 4
- Assessing Alterability
- **Previously**
- Measurement
- Measuring Quality
- Coupling and Cohesion
- Code Smells
- Large Class
- Key Points

Previously in COMPSCI701

- The closer the classes match concepts in the context schema (and design schema), the better the maintainability
- What does “match” mean? How can we tell that class “properly” matches the concept?
- One aspect to “match” is, **the expected number of objects get created**
- Example:

If have a class `Piece` representing the pieces on the board, then expect at most 24 `Piece` objects (12 one colour and 12 a different colour)

- Agenda
- Assignment 4
- Assessing Alterability
- Previously
- **Measurement**
- Measuring Quality
- Coupling and Cohesion
- Code Smells
- Large Class
- Key Points

Measurement

- **Measurement** is about assigning **numbers or symbols** to **attributes** of **entities** in such a way as to reliably **order** the entities with respect to the attribute (subject to measurement error limitations)
- The number or symbol is a **measurement**
- A **Metric** is the process that does the assignment
 - (Not to be confused with the “metric system” which is a set of internationally defined metrics¹)
- In order for a metric to be **valid**:
 - The process must give the “same” answer every time (within *measurement error*)
 - The process must assign number or symbols so that comparing the numbers or symbols gives the **same answer as direct comparison**

¹ (See www.stuff.co.nz/national/126432207/the-most-important-three-pieces-of-metal-in-new-zealand-explained)

- Agenda
- Assignment 4
- Assessing Alterability
- Previously
- Measurement
- **Measuring Quality**
- Coupling and Cohesion
- Code Smells
- Large Class
- Key Points

Measuring Quality

- A number of metrics have been proposed to measure “software quality” or “software maintainability” (including alterability)
- Very few have the property needed to be a valid measurement system

- Agenda
- Assignment 4
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- Coupling and Cohesion
- Code Smells
- Large Class
- Key Points

Coupling and Cohesion

- Concepts to characterise the quality of “modules” (e.g. classes) based on our intuition of how systems can be built more easily
- Ideas that were developed in the 1960s†
- If can measure them, then can improve “design quality” (at least with respect to some notions of quality)

† First formally published as: W. P. Stevens, G. J. Myers, and L. L. Constantine. Structured design. *IBM Systems Journal*, 13(2):115–139, 1974.

- Agenda
- Assignment 4
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- **Coupling and Cohesion**
- Code Smells
- Large Class
- Key Points

Definitions

Module a lexically contiguous sequence of program statements bounded by boundary elements, having an aggregate identifier†

Relationship A relationship exists between one module and another if that module cannot function correctly without the presence of the other

† Edward Yourdon and Larry L. Constantine. *Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design*. Prentice Hall, 1979.

Definitions

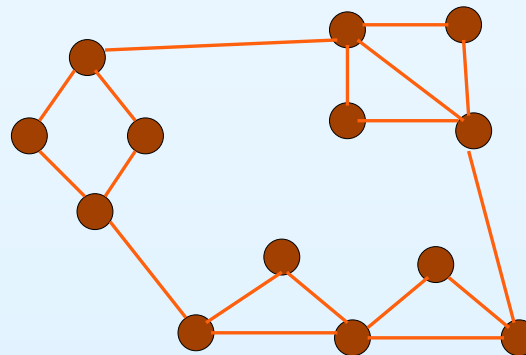
- Agenda
- Assignment 4
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- Coupling and Cohesion
- Code Smells
- Large Class
- Key Points

Coupling: ... the degree of interdependence between modules. ... a measure of the *strength* of interconnection (Yourdon and Constantine, 1979)

- The more the connections between one module and the rest, the harder to understand that module, the harder to re-use that module in another situation, the harder it is to isolate failures caused by faults in the module
- ⇒ The lower the coupling the “better”.

Cohesion: ... the extent to which its individual components are needed to perform the same task. ... how tightly bound or related [a module's] internal elements are to one another (Yourdon and Constantine, 1979)

- The less tightly bound the internal elements, the more disparate the parts to the module, the harder it is to understand
- ⇒ The higher the cohesion the “better”.



Definitions

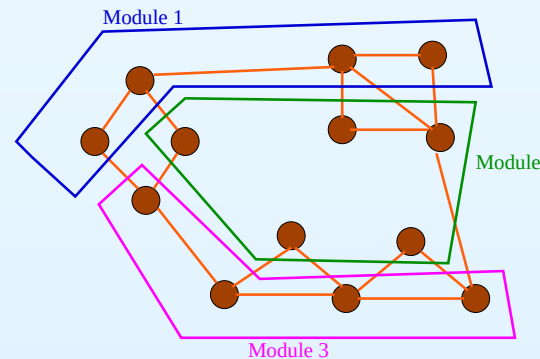
- Agenda
- Assignment 4
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- Coupling and Cohesion
- Code Smells
- Large Class
- Key Points

Coupling: ... the degree of interdependence between modules. ... a measure of the *strength* of interconnection (Yourdon and Constantine, 1979)

- The more the connections between one module and the rest, the harder to understand that module, the harder to re-use that module in another situation, the harder it is to isolate failures caused by faults in the module
- $\Rightarrow$ The lower the coupling the “better”.

Cohesion: ... the extent to which its individual components are needed to perform the same task. ... how tightly bound or related [a module's] internal elements are to one another (Yourdon and Constantine, 1979)

- The less tightly bound the internal elements, the more disparate the parts to the module, the harder it is to understand
- $\Rightarrow$ The higher the cohesion the “better”.



Definitions

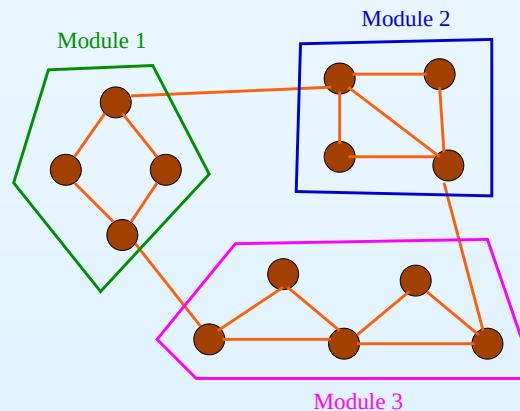
- Agenda
- Assignment 4
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- Coupling and Cohesion
- Code Smells
- Large Class
- Key Points

Coupling: ... the degree of interdependence between modules. ... a measure of the *strength* of interconnection (Yourdon and Constantine, 1979)

- The more the connections between one module and the rest, the harder to understand that module, the harder to re-use that module in another situation, the harder it is to isolate failures caused by faults in the module
- $\Rightarrow$ The lower the coupling the “better”.

Cohesion: ... the extent to which its individual components are needed to perform the same task. ... how tightly bound or related [a module's] internal elements are to one another (Yourdon and Constantine, 1979)

- The less tightly bound the internal elements, the more disparate the parts to the module, the harder it is to understand
- $\Rightarrow$ The higher the cohesion the “better”.



- Agenda
- Assignment 4
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- Coupling and Cohesion
- Code Smells
- Large Class
- Key Points

Measuring Coupling and Cohesion

- “Metrics” were proposed in the early days for procedural design
 - Coupling: Content, Common, Control, Stamp, Data
 - Cohesion: Coincidental, Logical, Temporal, Procedural, Communicational, Sequential, Functional, Informational
- More recently (1991^[1]) proposals have been made for object-oriented design. E.g.
 - Coupling Between Objects (CBO) — for class A, how many other classes does it with invoke a method or access a field on and vice versa
 - Lack of Cohesion of Methods (LCOM) — how many pairs of methods share the use of at least one field
- Also Martin^[2] and others^[3] have proposed some coupling metrics
 - $C_e^{[2]}$ — for A, the number of classes A depends on. (FAN-OUT^[3])
 - $C_a^{[2]}$ — for A, the number of classes that depend on A. (FAN-IN^[3])

[1] Shyam R. Chidamber and Chris F. Kemerer. Towards a metrics suite for object oriented design. In *Proceedings of 6th ACM Conference on Object-Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 197–211, 1991.

[2] Robert C. Martin. *OO Design Quality Metrics: An Analysis of Dependencies*, 1994.

[3] S. Henry, D. Kafura, Software structure metrics based on information flow, *IEEE Transactions on Software Engineering*. SE-7 (5) (September 1981) 510–518.

- Agenda
- Assignment 4
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- **Coupling and Cohesion**
- Code Smells
- Large Class
- Key Points

Example

Design A

```
class MovieLister {
    private CSVMovieFinder _finder = new CSVMovieFinder(...);

    public List<Movie> moviesDirectedBy(String dir) {
        List<Movie> allMovies = _finder.findAll();
        // iterate through 'allMovies' and find those with
        // 'dir' as directory
        ...
    }
}
```

None DI

Design B

```
interface MovieFinder {
    List<Movie> findAll();
}

class MovieLister {
    private MovieFinder _finder = null;
    public MovieLister(MovieFinder finder) {
        _finder = finder;
    }
    public List<Movie> moviesDirectedBy(String dir) {
        List<Movie> allMovies = _finder.findAll();
        // iterate through 'allMovies' and find those with
        // 'dir' as directory...
    }
}
```

DI

- Agenda
- Assignment 4
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- Coupling and Cohesion
- Code Smells
- Large Class
- Key Points

Coupling Measurements

- CBO(Design A MovieLister) = 1 (+ some number)—`CSVMovieFinder#CSVMovieFinder()`, `CSVMovieFinder#findAll()`
- CBO(Design B MovieLister) = 1 (+ same number)—`MovieFinder#findAll()`
- FAN-OUT(Design A MovieLister) = 3 (+ some number)—`CSVMovieFinder`, `List`, `Movie`
- FAN-OUT(Design B MovieLister) = 3 (+ same number)—`MovieFinder`, `List`, `Movie`
- “some number” — *we don’t have the complete design*
- We believe Design B is “better” than Design A, yet the measurements are the same
- This may be due to low precision (high measurement error)
- If so, then (assuming following DIP leads to better designs[‡]) CBO does not always distinguish between designs that use DIP and those that do not
- $\Rightarrow$ these metrics may (or may not be) reliable, but they are not very useful

[‡] PhD topic: Does use of DIP lead to “better”/ more maintainable/more comprehensive/more alterable/more testable designs?

Lack of Cohesion of Methods (LCOM)

- LCOM = The number of disjoint sets formed by the intersections of each pair of the sets of fields used by each method
- Class $\mathcal{C}$ with
 - k fields $f_1, f_2, f_3, \dots, f_k$
 - n public methods $m_1, m_2, m_3, \dots, m_n$.

$$\mathcal{I}_i = \{f_l : f_l \text{ is used by } m_i\}.$$

$\mathcal{N}$ — number of different possible pairs of methods ($\mathcal{N} = \frac{n(n-1)}{2}$).

$$\mathcal{P} = |\{(m_i, m_j) : i < j \text{ and } \mathcal{I}_i \cap \mathcal{I}_j = \phi\}|$$

$$\mathcal{Q} = |\{(m_i, m_j) : i < j \text{ and } \mathcal{I}_i \cap \mathcal{I}_j \neq \phi\}|.$$

$$(\mathcal{N} = \mathcal{P} + \mathcal{Q})$$

$$\text{LCOM}_1(\text{OOPSLA1991}^{[1]}) = \mathcal{P}$$

$$\text{LCOM}_2(\text{TSE1994}^{[4]}) = \max(0, \mathcal{P} - \mathcal{Q})$$

[1] Shyam R. Chidamber and Chris F. Kemerer. Towards a metrics suite for object oriented design. In *Proceedings of 6th ACM Conference on Object-Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 197–211, 1991.

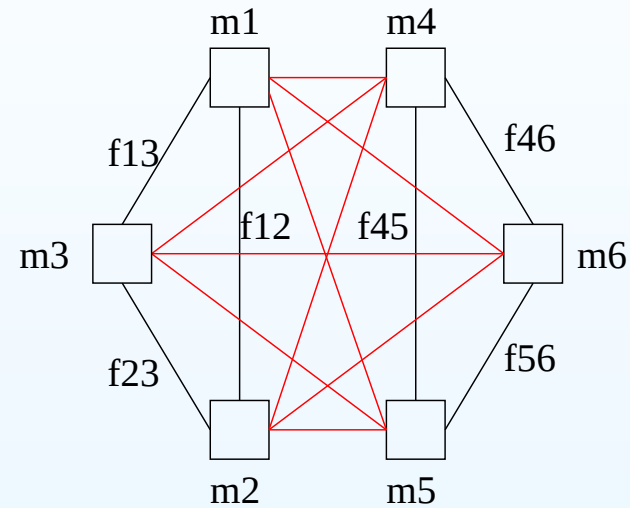
[4] Shyam R. Chidamber and Chris F. Kemerer. A Metrics Suite for Object Oriented Design. **IEEE Transactions on Software Engineering**, 20(6):476-493, June 1994

- Agenda
- Assignment 4
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- **Coupling and Cohesion**
- Code Smells
- Large Class
- Key Points

Variants

```
public class A {
    private String f12, f23, f13, f45, f56, f46;

    public aMethod1() {
        f12 = f13;
    }
    public aMethod2() {
        f12 = f23;
    }
    public aMethod3() {
        f23 = f13;
    }
    public aMethod4() {
        f45 = f46;
    }
    public aMethod5() {
        f45 = f56;
    }
    public aMethod6() {
        f46 = f56;
    }
}
```



2 connected
components
6 black edges
9 red edges

LCOM 1 = P (red edges) (OOPSLA 1991)

LCOM 2 = $\text{Max}(P - Q, 0)$ (TSE 1994)

LCOM 3 = # connected components[5]

[5] W. Li and S. Henry. *Object-oriented metrics that predict maintainability*. *J. Syst. Softw.*, 23(2):111–122, 1993.

- Agenda
- Assignment 4
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- Coupling and Cohesion
- Code Smells
- Large Class
- Key Points

LCOM

```
public PersonDetails {
    private String _firstname;
    private String _surname;
    private String _street;
    private String _city;
     $\mathcal{I}_1 = \{ \}$ 
    public PersonDetails() {}
     $\mathcal{I}_2 = \{ \_firstname, \_surname \}$ 
    public setName(String f, String s) {
        _firstname = f; _surname = s;
    }
    public void printName() {
        System.out.println(_firstname + " " + _surname);
    }
     $\mathcal{I}_3 = \{ \_street, \_city \}$ 
    public setAddress(String st, String c) {
        _street = st; _city = c;
    }
     $\mathcal{I}_4 = \{ \_street, \_city \}$ 
    public void printAddress() {
        System.out.println(_street);
        System.out.println(_city);
    }
     $\mathcal{I}_5 = \{ \_firstname, \_surname \}$ 
}
```

- LCOM 1 = 8, LCOM 2 = (8-2) = 6, LCOM 3 = 2
- Observations: This class may change because of changes to the name or changes to the address (two reasons to change); it has (at least) two responsibilities — managing the name and managing the address

Alterability

- Agenda
- Assignment 4
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- Coupling and Cohesion
- Code Smells
- Large Class
- Key Points

- If class A is coupled to class B, there is a chance that a change to B will require a change to A. If A is not coupled to B, then no change to B can lead to a change in A.

So if there is coupling, there is a chance C will be higher, so higher $\mathcal{CR}$

- For class A, the more other classes it is coupled to, there more likely a change case will require A to change, so the more likely C will be higher (and so $\mathcal{CR}$)
- A class with low cohesion typically means the class is not a good match to a concept in the context schema. If it matches multiple concepts, then the more likely C will be higher (and so $\mathcal{CR}$)

- Agenda
- Assignment 4
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- Coupling and Cohesion
- Code Smells
- Large Class
- Key Points

Maintainability

- If class A is coupled to class B, then understanding A may require understanding something of B, so more work to understand so less efficient to understand
- If class A is coupled to class B, then testing A requires doing something with B, so more work to understand so less efficient to test

- Agenda
- Assignment 4
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- Coupling and Cohesion
- Code Smells
- Large Class
- Key Points

Evaluation

- Coupling and Cohesion have appealing intuition — everyone “knows” what they mean, and they appear to tell us something about alterability
- But there are no reliable (or at least useful) metrics for them
- Even if we had measurements for them, it is not clear how to use such measurements to change the design—“reduce coupling” is not much better than “improve design”

- Agenda
- Assignment 4
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- Coupling and Cohesion
- **Code Smells**
- Large Class
- Key Points

Code Smells

- Code smell - characteristic of design that indicate potential design quality problems
- *Heuristics* — not hard and fast rules but just an indication that there may be a problem and so requires a closer look
- From Martin Fowler “Refactoring: Improving the design of existing code” Addison-Wesley, 1999 (Code Smell chapter co-written with Kent Beck)

- Agenda
- Assignment 4
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- Coupling and Cohesion
- **Code Smells**
- Large Class
- Key Points

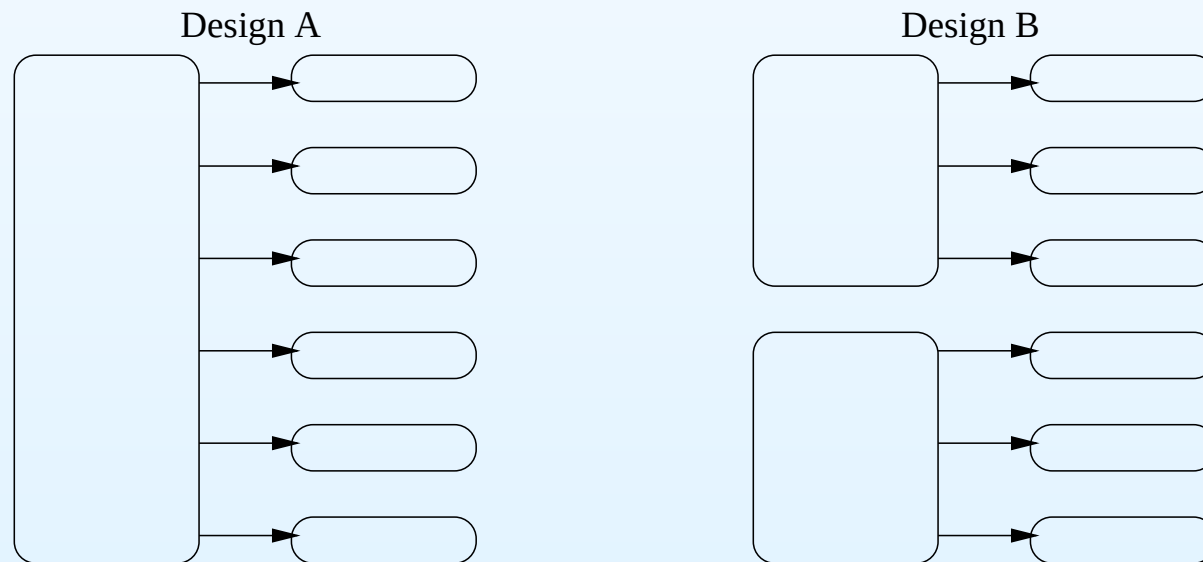
Code Smell Examples

- Duplicate Code
- Long Method
- Large Class
- Long Parameter List
- Speculative Generality
- Data class
- Comments as deodorant
- Temporary Field
- (and many more)

Large Class

- When is a class “large”?
 - Is 100 LOC “large”? If the whole system is 10,000 LOC then maybe not.
 - But if the whole system is 300 LOC then maybe

⇒ do not use LOC (or any absolute measurement) when defining Large Class
- The larger the class, the more of the total code it occupies, so the **more change cases it will need to be changed for**, increasing $CR \Rightarrow$ lower alterability
- The larger the class, the less likely it can be following SRP
- The larger the class, the higher the likely coupling
- The larger the class, the lower the likely cohesion



- Agenda
- Assignment 4
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- Coupling and Cohesion
- Code Smells
- **Large Class**
- Key Points

Related Smells

- Long Method — the longer a method is, the larger the class it is in must be ⇒ **Large Class**
- Long Parameter List — the more parameters, the longer the method is likely to be **Long Method**

- Agenda
- Assignment 4
- Assessing Alterability
- Previously
- Measurement
- Measuring Quality
- Coupling and Cohesion
- Code Smells
- Large Class
- Key Points

Key Points

- Measurement of software is really really complicated, so be suspicious of any proposed software metrics, especially any that claim to measure quality, because (a) they don't and (b) they're not useful for making design decisions
- Coupling and Cohesion are frequently mentioned as useful attributes of design quality, but while the intuition seems reasonable it is difficult to base decisions on them.
- If any of the design advice has any value, there **must be relationships between each piece of advice**